VIT-AP UNIVERSITY,ANDHRA PRADESH

Lab Sheet 6:MongoDB Basic commands

Branch: B.Tech(C.S.E) **Date:** 11/03/2026

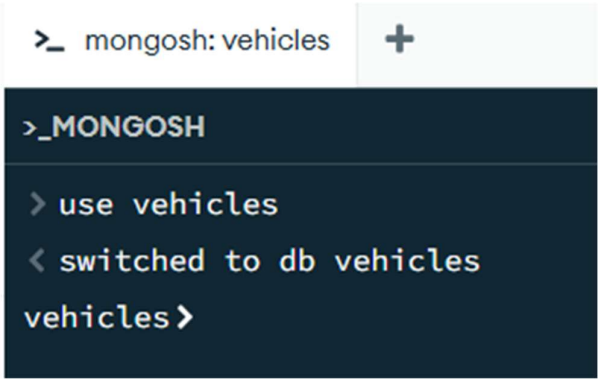
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE

Student name: Mandadi Varun **Reg. no.:**23BCE9738

1. Use MongoDB to implement the following DB operations

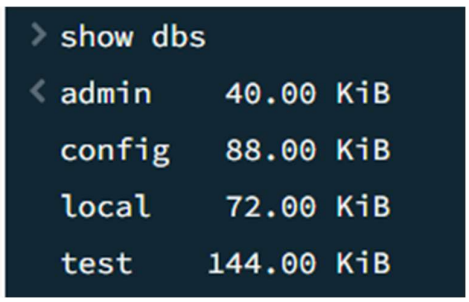
1. Create a database called 'vehicles' and *write* a MongoDB query to select database as "vehicles".

   Query:



2. Write a MongoDB query to display all the databases.

   Query:



3. Create a collection called ‘two_wheelers’. (use capping) and Create a collection called ‘four_wheelers’.

   Query:

```
>_MONGOSH
> db.createCollection("two_wheelers", { capped: true, size: 100000, max: 100 })
< { ok: 1 }
> db.createCollection("four_wheelers")
< { ok: 1 }
test>
```

4. Add 5 two-wheeler details to the collection named ‘two_wheelers’. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

Query:

```
>_MONGOSH
  {
  bike_name: "Yamaha R15",
  model: "gear",
  category: "150cc",
  colors_available: ["sport red","black"],
  manufacturer: "Yamaha",
  performance: 9,
  timestamp: new Date("2023-02-20"),
  price: 180000
  },
  {
  bike_name: "Bajaj Pulsar",
  model: "gear",
  category: "150cc",
  colors_available: ["red","black"],
  manufacturer: "Bajaj",
  performance: 8,
  timestamp: new Date("2020-09-10"),
  price: 120000
  },
```

```
>_MONGOSH
> db.two_wheelers.insertMany([
  {
  bike_name: "Honda Shine",
  model: "gear",
  category: "125cc",
  colors_available: ["red","black","blue"],
  manufacturer: "Honda",
  performance: 8,
  timestamp: new Date("2022-05-10"),
  price: 90000
  },
  {
  bike_name: "TVS Jupiter",
  model: "gearless",
  category: "125cc",
  colors_available: ["white","black","blue"],
  manufacturer: "TVS",
  performance: 7,
  timestamp: new Date("2021-07-15"),
  price: 85000
  },
```

```
  {
  bike_name: "Hero Splendor",
  model: "gear",
  category: "100cc",
  colors_available: ["black","blue"],
  manufacturer: "Hero",
  performance: 7,
  timestamp: new Date("2019-03-12"),
  price: 75000
  }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b13e024c92833db957ed9b'),
      '1': ObjectId('69b13e024c92833db957ed9c'),
      '2': ObjectId('69b13e024c92833db957ed9d'),
      '3': ObjectId('69b13e024c92833db957ed9e'),
      '4': ObjectId('69b13e024c92833db957ed9f')
    }
  }
test>
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

Query:

```
>_MONGOSH
> db.four_wheelers.insertMany([
  {
  vehicle_name: "Maruti Swift",
  model: "own",
  category: "car",
  variants: ["vxi","zxi","petrol"],
  manufacturer: "Maruti",
  performance: 8,
  timestamp: new Date("2022-06-10"),
  price: 700000
  },
  {
  vehicle_name: "Tata Ace",
  model: "commercial",
  category: "mini truck",
  variants: ["diesel"],
  manufacturer: "Tata",
  performance: 7,
  timestamp: new Date("2020-04-12"),
  price: 500000
  },
```

```
>_MONGOSH
  {
  vehicle_name: "Ashok Leyland Bus",
  model: "commercial",
  category: "bus",
  variants: ["diesel"],
  manufacturer: "Ashok Leyland",
  performance: 8,
  timestamp: new Date("2019-11-20"),
  price: 2500000
  },
  {
  vehicle_name: "Mahindra Bolero",
  model: "own",
  category: "car",
  variants: ["diesel","zxi"],
  manufacturer: "Mahindra",
  performance: 7,
  timestamp: new Date("2021-01-18"),
  price: 950000
  },
```

```
  {
  vehicle_name: "Eicher Heavy Truck",
  model: "commercial",
  category: "heavy truck",
  variants: ["diesel"],
  manufacturer: "Eicher",
  performance: 8,
  timestamp: new Date("2018-08-25"),
  price: 3000000
  }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b13f214c92833db957eda0'),
      '1': ObjectId('69b13f214c92833db957eda1'),
      '2': ObjectId('69b13f214c92833db957eda2'),
      '3': ObjectId('69b13f214c92833db957eda3'),
      '4': ObjectId('69b13f214c92833db957eda4')
    }
  }
test>
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

Query:

```
>_MONGOSH
> db.two_wheelers.find()
< {
    _id: ObjectId('69b1378e2fc689f82b05e157'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [
      'red',
      'black',
      'blue'
    ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: 2022-05-10T00:00:00.000Z,
    price: 90000
  }
  {
    _id: ObjectId('69b1378e2fc689f82b05e158'),
    bike_name: 'TVS Jupiter',
    model: 'gearless',
    category: '125cc',
    colors_available: [
      'white',
      'black',
      'blue'
    ],
    manufacturer: 'TVS',
    performance: 7,
    timestamp: 2021-07-15T00:00:00.000Z,
```

```
>_MONGOSH
    price: 85000
  }
  {
    _id: ObjectId('69b1378e2fc689f82b05e159'),
    bike_name: 'Yamaha R15',
    model: 'gear',
    category: '150cc',
    colors_available: [
      'sport red',
      'black'
    ],
    manufacturer: 'Yamaha',
    performance: 9,
    timestamp: 2023-02-20T00:00:00.000Z,
    price: 180000
  }
  {
    _id: ObjectId('69b1378e2fc689f82b05e15a'),
    bike_name: 'Bajaj Pulsar',
    model: 'gear',
    category: '150cc',
    colors_available: [
      'red',
      'black'
    ],
    manufacturer: 'Bajaj',
    performance: 8,
    timestamp: 2020-09-10T00:00:00.000Z,
    price: 120000
```

```
>_MONGOSH
  }
  {
    _id: ObjectId('69b1378e2fc689f82b05e15b'),
    bike_name: 'Hero Splendor',
    model: 'gear',
    category: '100cc',
    colors_available: [
      'black',
      'blue'
    ],
    manufacturer: 'Hero',
    performance: 7,
    timestamp: 2019-03-12T00:00:00.000Z,
    price: 75000
  }
  {
    _id: ObjectId('69b13e024c92833db957ed9b'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [
      'red',
      'black',
      'blue'
    ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: 2022-05-10T00:00:00.000Z,
    price: 90000
```

```
>_MONGOSH
  }
  {
    _id: ObjectId('69b13e024c92833db957ed9c'),
    bike_name: 'TVS Jupiter',
    model: 'gearless',
    category: '125cc',
    colors_available: [
      'white',
      'black',
      'blue'
    ],
    manufacturer: 'TVS',
    performance: 7,
    timestamp: 2021-07-15T00:00:00.000Z,
    price: 85000
  }
  {
    _id: ObjectId('69b13e024c92833db957ed9d'),
    bike_name: 'Yamaha R15',
    model: 'gear',
    category: '150cc',
    colors_available: [
      'sport red',
      'black'
    ],
    manufacturer: 'Yamaha',
    performance: 9,
    timestamp: 2023-02-20T00:00:00.000Z,
    price: 180000
```

```
{
  _id: ObjectId('69b13e024c92833db957ed9e'),
  bike_name: 'Bajaj Pulsar',
  model: 'gear',
  category: '150cc',
  colors_available: [
    'red',
    'black'
  ],
  manufacturer: 'Bajaj',
  performance: 8,
  timestamp: 2020-09-10T00:00:00.000Z,
  price: 120000
}
{
  _id: ObjectId('69b13e024c92833db957ed9f'),
  bike_name: 'Hero Splendor',
  model: 'gear',
  category: '100cc',
  colors_available: [
    'black',
    'blue'
  ],
  manufacturer: 'Hero',
  performance: 7,
  timestamp: 2019-03-12T00:00:00.000Z,
  price: 75000
}
```


>_MONGOSH

```
> db.four_wheelers.find()
< {
    _id: ObjectId('69b137a62fc689f82b05e15c'),
    vehicle_name: 'Maruti Swift',
    model: 'own',
    category: 'car',
    variants: [
      'vxi',
      'zxi',
      'petrol'
    ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: 2022-06-10T00:00:00.000Z,
    price: 700000
  }
  {
    _id: ObjectId('69b137a62fc689f82b05e15d'),
    vehicle_name: 'Tata Ace',
    model: 'commercial',
    category: 'mini truck',
    variants: [
      'diesel'
    ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: 2020-04-12T00:00:00.000Z,
    price: 500000
  }
```

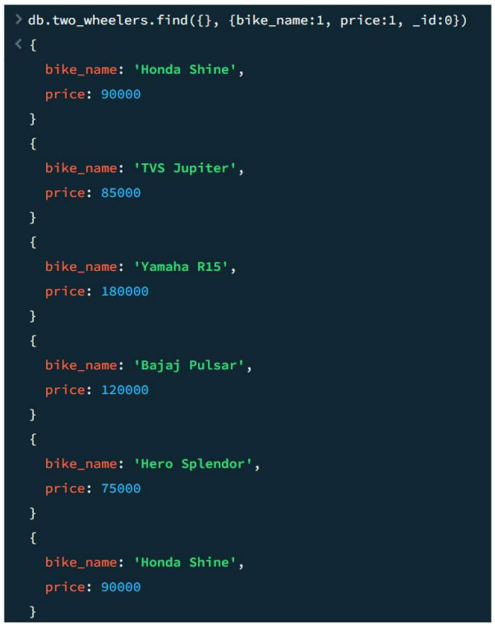

```
{
  _id: ObjectId('69b137a62fc689f82b05e15e'),
  vehicle_name: 'Ashok Leyland Bus',
  model: 'commercial',
  category: 'bus',
  variants: [
    'diesel'
  ],
  manufacturer: 'Ashok Leyland',
  performance: 8,
  timestamp: 2019-11-20T00:00:00.000Z,
  price: 2500000
}
{
  _id: ObjectId('69b137a62fc689f82b05e15f'),
  vehicle_name: 'Mahindra Bolero',
  model: 'own',
  category: 'car',
  variants: [
    'diesel',
    'zxi'
  ],
  manufacturer: 'Mahindra',
  performance: 7,
  timestamp: 2021-01-18T00:00:00.000Z,
  price: 950000
}
```



```
{
  _id: ObjectId('69b137a62fc689f82b05e160'),
  vehicle_name: 'Eicher Heavy Truck',
  model: 'commercial',
  category: 'heavy truck',
  variants: [
    'diesel'
  ],
  manufacturer: 'Eicher',
  performance: 8,
  timestamp: 2018-08-25T00:00:00.000Z,
  price: 3000000
}
{
  _id: ObjectId('69b13f214c92833db957eda0'),
  vehicle_name: 'Maruti Swift',
  model: 'own',
  category: 'car',
  variants: [
    'vxi',
    'zxi',
    'petrol'
  ],
  manufacturer: 'Maruti',
  performance: 8,
  timestamp: 2022-06-10T00:00:00.000Z,
  price: 700000
}
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

Query:



```
  {
    vehicle_name: 'Mahindra Bolero',
    price: 950000
  }
  {
    vehicle_name: 'Eicher Heavy Truck',
    price: 3000000
  }
  {
    vehicle_name: 'Maruti Swift',
    price: 700000
  }
  {
    vehicle_name: 'Tata Ace',
    price: 500000
  }
  {
    vehicle_name: 'Ashok Leyland Bus',
    price: 2500000
  }
  {
    vehicle_name: 'Mahindra Bolero',
    price: 950000
  }
  {
    vehicle_name: 'Eicher Heavy Truck',
    price: 3000000
  }
test>
```

8. Write a MongoDB query to display two_wheelers from a particular company

Query:

```
> db.two_wheelers.find({manufacturer:"Honda"})
< {
    _id: ObjectId('69b1378e2fc689f82b05e157'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [
      'red',
      'black',
      'blue'
    ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: 2022-05-10T00:00:00.000Z,
    price: 90000
  }
```

```
{
  _id: ObjectId('69b13e024c92833db957ed9b'),
  bike_name: 'Honda Shine',
  model: 'gear',
  category: '125cc',
  colors_available: [
    'red',
    'black',
    'blue'
  ],
  manufacturer: 'Honda',
  performance: 8,
  timestamp: 2022-05-10T00:00:00.000Z,
  price: 90000
}
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

Query:

```
> db.four_wheelers.find({variants:"diesel"})
< {
    _id: ObjectId('69b137a62fc689f82b05e15d'),
    vehicle_name: 'Tata Ace',
    model: 'commercial',
    category: 'mini truck',
    variants: [
      'diesel'
    ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: 2020-04-12T00:00:00.000Z,
    price: 500000
  }
```

```
{
  _id: ObjectId('69b137a62fc689f82b05e15e'),
  vehicle_name: 'Ashok Leyland Bus',
  model: 'commercial',
  category: 'bus',
  variants: [
    'diesel'
  ],
  manufacturer: 'Ashok Leyland',
  performance: 8,
  timestamp: 2019-11-20T00:00:00.000Z,
  price: 2500000
}
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

Query:

```
> db.two_wheelers.find(
  {performance:{$gt:5}},
  {bike_name:1, category:1, manufacturer:1, _id:0}
  )
< {
    bike_name: 'Honda Shine',
    category: '125cc',
    manufacturer: 'Honda'
  }
  {
    bike_name: 'TVS Jupiter',
    category: '125cc',
    manufacturer: 'TVS'
  }
```

```
> db.four_wheelers.find(
  {performance:{$gt:5}},
  {vehicle_name:1, category:1, manufacturer:1, _id:0}
  )
< {
    vehicle_name: 'Maruti Swift',
    category: 'car',
    manufacturer: 'Maruti'
  }
  {
    vehicle_name: 'Tata Ace',
    category: 'mini truck',
    manufacturer: 'Tata'
  }
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called ‘animal’ and *write* a MongoDB query to select database as ‘animal’.

   Query:

```
>_MONGOSH
> use animal
< switched to db animal
animal>
```

2. Write a MongoDB query to display all the databases.

   Query:

```
> show dbs
< admin    40.00 KiB
  config   96.00 KiB
  local    72.00 KiB
  test    144.00 KiB
animal>
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

   Query:

```
> db.createCollection("wild_animals", { capped: true, size: 100000, max: 100 })
< { ok: 1 }
> db.createCollection("domestic_animals")
< { ok: 1 }
> show collections
< domestic_animals
  wild_animals
animal>
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

   Query:

```
> db.wild_animals.insertMany([
  {
  animal_name: "Lion",
  nature: "harm",
  favorite_foods: ["meat","deer","rabbit"],
  care_taker_name: "Ravi",
  life_span: 14,
  timestamp: new Date("2020-05-10"),
  expenses: 50000
  },
  {
  animal_name: "Tiger",
  nature: "harm",
  favorite_foods: ["meat","deer"],
  care_taker_name: "Suresh",
  life_span: 15,
  timestamp: new Date("2019-03-12"),
  expenses: 55000
  },
  {
  animal_name: "Elephant",
  nature: "harmless",
  favorite_foods: ["grass","fruits"],
  care_taker_name: "Mahesh",
  life_span: 60,
  timestamp: new Date("2018-08-25"),
  expenses: 70000
  },
  {
  animal_name: "Leopard",
  nature: "harm",
  favorite_foods: ["meat","rabbit"],
  care_taker_name: "Ravi",
  life_span: 13,
  timestamp: new Date("2021-01-15"),
  expenses: 45000
  },
  {
  animal_name: "Deer",
  nature: "harmless",
  favorite_foods: ["grass","leaves"],
  care_taker_name: "Kiran",
  life_span: 10,
  timestamp: new Date("2022-06-18"),
  expenses: 20000
  }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b162561ac4c7d32a9318c6'),
      '1': ObjectId('69b162561ac4c7d32a9318c7'),
      '2': ObjectId('69b162561ac4c7d32a9318c8'),
      '3': ObjectId('69b162561ac4c7d32a9318c9'),
      '4': ObjectId('69b162561ac4c7d32a9318ca')
    }
  }
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

Query:

```
> db.domestic_animals.insertMany([
  {
  animal_name: "Dog",
  gender: "male",
  favorite_foods: ["meat","bones"],
  animal_petname: "Tommy",
  life_span: 12,
  timestamp: new Date("2021-04-10"),
  expenses: 8000
  },
  {
  animal_name: "Cat",
  gender: "female",
  favorite_foods: ["fish","milk"],
  animal_petname: "Kitty",
  life_span: 14,
  timestamp: new Date("2020-07-15"),
  expenses: 6000
  },
  {
  animal_name: "Cow",
  gender: "female",
  favorite_foods: ["grass","grains"],
  animal_petname: "Lakshmi",
  life_span: 20,
  timestamp: new Date("2019-05-20"),
  expenses: 15000
  },
  {
  animal_name: "Goat",
  gender: "male",
  favorite_foods: ["grass","leaves"],
  animal_petname: "Raju",
  life_span: 15,
  timestamp: new Date("2022-02-11"),
  expenses: 7000
  },
  {
  animal_name: "Rabbit",
  gender: "female",
  favorite_foods: ["carrot","vegetables"],
  animal_petname: "Bunny",
  life_span: 9,
  timestamp: new Date("2023-01-05"),
  expenses: 5000
  }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b162bb1ac4c7d32a9318cb'),
      '1': ObjectId('69b162bb1ac4c7d32a9318cc'),
      '2': ObjectId('69b162bb1ac4c7d32a9318cd'),
      '3': ObjectId('69b162bb1ac4c7d32a9318ce'),
      '4': ObjectId('69b162bb1ac4c7d32a9318cf')
    }
  }
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

Query:

```
> db.wild_animals.find()
< {
    _id: ObjectId('69b162561ac4c7d32a9318c6'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [
      'meat',
      'deer',
      'rabbit'
    ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: 2020-05-10T00:00:00.000Z,
    expenses: 50000
  }
```

```
> db.domestic_animals.find()
< {
    _id: ObjectId('69b162bb1ac4c7d32a9318cb'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [
      'meat',
      'bones'
    ],
    animal_petname: 'Tommy',
    life_span: 12,
    timestamp: 2021-04-10T00:00:00.000Z,
    expenses: 8000
  }
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

Query:

```
> db.wild_animals.find({}, {animal_name:1, expenses:1, _id:0})
< {
    animal_name: 'Lion',
    expenses: 50000
  }
  {
    animal_name: 'Tiger',
    expenses: 55000
  }
  {
    animal_name: 'Elephant',
    expenses: 70000
  }
  {
    animal_name: 'Leopard',
    expenses: 45000
  }
  {
    animal_name: 'Deer',
    expenses: 20000
  }
```

```
> db.domestic_animals.find({}, {animal_name:1, expenses:1, _id:0})
< {
    animal_name: 'Dog',
    expenses: 8000
  }
  {
    animal_name: 'Cat',
    expenses: 6000
  }
  {
    animal_name: 'Cow',
    expenses: 15000
  }
  {
    animal_name: 'Goat',
    expenses: 7000
  }
  {
    animal_name: 'Rabbit',
    expenses: 5000
  }
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

Query:

```
> db.domestic_animals.find({life_span:12})
< {
    _id: ObjectId('69b162bb1ac4c7d32a9318cb'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [
      'meat',
      'bones'
    ],
    animal_petname: 'Tommy',
    life_span: 12,
    timestamp: 2021-04-10T00:00:00.000Z,
    expenses: 8000
  }
animal>
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

Query:

```
> db.wild_animals.find({care_taker_name:"Ravi"})
< {
    _id: ObjectId('69b162561ac4c7d32a9318c6'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [
      'meat',
      'deer',
      'rabbit'
    ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: 2020-05-10T00:00:00.000Z,
    expenses: 50000
  }
```

```
  {
    _id: ObjectId('69b162561ac4c7d32a9318c9'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [
      'meat',
      'rabbit'
    ],
    care_taker_name: 'Ravi',
    life_span: 13,
    timestamp: 2021-01-15T00:00:00.000Z,
    expenses: 45000
  }
animal>
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

Query:

```
> db.wild_animals.find(
  {life_span:{$gt:5}},
  {animal_name:1, favorite_foods:1, expenses:1, _id:0}
  )
< {
    animal_name: 'Lion',
    favorite_foods: [
      'meat',
      'deer',
      'rabbit'
    ],
    expenses: 50000
  }
  {
    animal_name: 'Tiger',
    favorite_foods: [
      'meat',
      'deer'
    ],
    expenses: 55000
  }
```

```
  {
    animal_name: 'Elephant',
    favorite_foods: [
      'grass',
      'fruits'
    ],
    expenses: 70000
  }
  {
    animal_name: 'Leopard',
    favorite_foods: [
      'meat',
      'rabbit'
    ],
    expenses: 45000
  }
  {
    animal_name: 'Deer',
    favorite_foods: [
      'grass',
      'leaves'
    ],
    expenses: 20000
  }
animal>
```

```
> db.domestic_animals.find(
  {life_span:{$gt:5}},
  {animal_name:1, favorite_foods:1, expenses:1, _id:0}
  )
< {
    animal_name: 'Dog',
    favorite_foods: [
      'meat',
      'bones'
    ],
    expenses: 8000
  }
  {
    animal_name: 'Cat',
    favorite_foods: [
      'fish',
      'milk'
    ],
    expenses: 6000
  }
```

```
  {
    animal_name: 'Cow',
    favorite_foods: [
      'grass',
      'grains'
    ],
    expenses: 15000
  }
  {
    animal_name: 'Goat',
    favorite_foods: [
      'grass',
      'leaves'
    ],
    expenses: 7000
  }
  {
    animal_name: 'Rabbit',
    favorite_foods: [
      'carrot',
      'vegetables'
    ],
    expenses: 5000
  }
animal>
```